

Discipline: Object-Oriented Programming: Java (OOP: Java)

Lecture 3. Classes and Methods

Лекция 3. КЛАССЫ И МЕТОДЫ

Classes and Methods in Java

A class defines data and behavior. An object is a working instance of a class.

```
public class Student {  
    String name;        // field  
    int age;            // field  
    double gpa;         // field  
  
    void displayInfo() {  
        System.out.println(name + " | " + age + " | " + gpa);  
    }  
}  
  
Student student1 = new Student();  
student1.name = "Anna";  
student1.age = 20;  
student1.gpa = 3.7;  
student1.displayInfo();
```

CLASS → defines structure and behavior

OBJECT → a specific instance of a class

FIELDS → store object data

METHODS → define object actions

Class → Object → Data → Method → Result

Think about it: How many Student objects can we create from one Student class?

Learning Objectives

By the end of this lecture, you will be able to:

- 01 **Explain** the difference between a class and an object.
- 02 **Create** a Java class with fields and methods.
- 03 **Create and use** objects with the new operator.
- 04 **Write** methods with parameters and return values.
- 05 **Use** constructors and the this keyword.
- 06 **Apply** encapsulation with private fields, getters, setters, and validation.

Final outcome: Design, create, run, and extend a simple Java class in IntelliJ IDEA.

What Is a Class?

A class is a blueprint that defines the data (fields) and behavior (methods) of objects.

A class describes:

FIELDS

What data does an object store?

METHODS

What actions can an object perform?

```
class Student {  
    // fields  
    String name;  
    int age;  
  
    // method  
    void study() {  
        System.out.println(name + " is studying.");  
    }  
}
```

Class Student → fields: name, age + method: study() → Student objects

Question: Which parts of the code describe the state, and which part describes the behavior?

What Is an Object?

An object is a specific instance of a class. Each object has its own data and can use the methods defined by the class.

CLASS
Student
name
age
gpa
displayInfo()

new
→

OBJECT
student1
name = "Anna"
age = 20
gpa = 3.7

OBJECT
student2
name = "Alex"
age = 21
gpa = 3.4

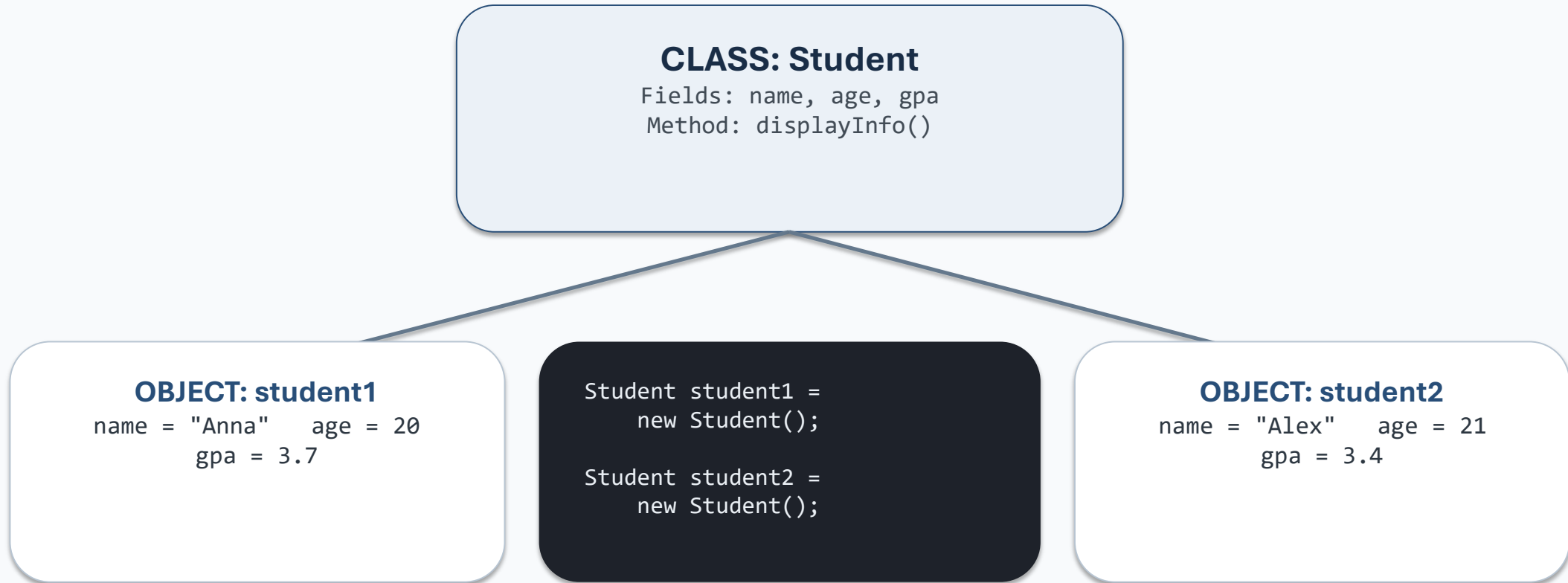
```
Student student1 = new Student();    Student student2 = new Student();
```

One class → many objects. Each object has its own state.

Question: What is the same for student1 and student2, and what is different?

Class vs Object

A class is a template. Objects are individual instances created from that class.



Same class → same structure and methods | Different objects → different field values

Structure of a Java Class

A Java class combines data (fields) and behavior (methods) in one structure.

```
public class Student {  
  
    // FIELDS – state / data  
    String name;  
    int age;  
    double gpa;  
  
    // METHOD – behavior  
    void displayInfo() {  
        System.out.println(name);  
        System.out.println(age);  
        System.out.println(gpa);  
    }  
}
```

Question: Which lines store data, and which lines define behavior?

1 CLASS HEADER

public class Student

Defines the class name.

2 FIELDS

name, age, gpa

Store the state of each object.

3 METHODS

displayInfo()

Define what an object can do.

CLASS = FIELDS + METHODS

Fields — The State of an Object

Fields are variables declared inside a class. They store the current data of each object.

```
class Student {  
    String name;  
    int age;  
    double gpa;  
}  
  
Student student1 = new Student();  
student1.name = "Anna";  
student1.age = 20;  
student1.gpa = 3.7;
```

FIELDS = the current STATE of an object

Object state after assignment

```
student1  
name  = "Anna"  
age   = 20  
gpa   = 3.7
```

Fields can change during program execution:

```
student1.gpa = 3.9;  
System.out.println(student1.gpa);    // 3.9
```

Predict before Run: What will be printed after student1.gpa = 3.9?

Field Data Types

Each field has a data type. The type defines what kind of value the field can store.

```
class Student {  
    String name;        // text  
    int age;             // whole number  
    double gpa;          // decimal number  
    boolean active;      // true / false  
}
```

```
Student student1 = new Student();  
student1.name = "Anna";  
student1.age = 20;  
student1.gpa = 3.7;  
student1.active = true;
```

String name = "Anna"
text / characters

int age = 20
whole numbers

double gpa = 3.7
decimal numbers

boolean active = true
true or false

student1.age = "20"; ← Will this compile?

Predict before Run: Which value is invalid for its field type — 21, 3.9, false, or "21" for int age?

Creating an Object — the new Operator

The new operator creates a new object in memory and returns a reference to it.

```
Student student1 = new Student();
```

```
student1.name = "Anna";  
student1.age = 20;  
student1.gpa = 3.7;
```

Read the statement step by step

Student → type of the reference

student1 → reference variable

= → assignment

new → creates a new object

1. Declare

Student student1

2. Create

new Student()

3. Reference

student1 → object

4. Use

student1.name

```
Student student2 = new Student(); // another independent object
```

new → creates a **NEW** object

Predict: If we execute new Student() twice, do we get one object or two different objects?

Accessing Object Fields — the Dot Operator (.)

We use the dot operator (.) to access a field through an object reference.

`objectReference.fieldName` → `student1.name`

```
Student student1 = new Student();

// WRITE values to fields
student1.name = "Anna";
student1.age = 20;
student1.gpa = 3.7;

// READ values from fields
System.out.println(student1.name);
System.out.println(student1.gpa);
```

student1 → object reference

. → dot operator

name → field name

= → assign a value

WRITE: `student1.age = 20;`
READ: `System.out.println(student1.age);`

Predict before Run: What will `student1.name` print after `student1.name = "Anna"`?

Several Objects of One Class

Definition. One class can be used to create many independent objects. Each object has its own state (field values), while all objects share the structure. **Changing the fields of one object does not automatically change the fields of another object.**

```
class Student {  
    String name;  
    int age;  
    double gpa;  
    String major;        // NEW field  
    int completedCredits; // NEW field  
  
    void displayProfile() {        // NEW method  
        System.out.println(name + " | " + major);  
        System.out.println("GPA: " + gpa);  
    }  
  
    boolean canGraduate() {        // NEW method  
        return completedCredits >= 240;  
    }  
}
```

Three objects → three different states

student1: Anna, age 20, GPA 3.8
Computer Science | credits = 210

student2: Alex, age 21, GPA 3.2
Software Eng. | credits = 245

student3: Mira, age 19, GPA 3.9
Data Science | credits = 180

```
student2.displayProfile();  
System.out.println(student2.canGraduate()); // true
```

Think: Which student can graduate? What changes if `student1.completedCredits` becomes 250?

What Is a Method? — Object Behavior

Definition. A method is a named block of code inside a class that describes an action or behavior. **An object calls a method to perform that action.** A method may read or change fields, receive parameters, perform calculations, and return a result.

```
class Student {
    String name;
    double gpa;
    int completedCredits;

    void study() {           // action
        System.out.println(name + " is studying.");
    }

    void addCredits(int credits) { // changes state
        completedCredits += credits;
    }

    double calculateProgress() { // returns result
        return completedCredits / 240.0 * 100;
    }
}
```

Predict: If completedCredits = 210, what happens after addCredits(10)?

Method anatomy

returnType methodName(parameters)

void → no value is returned

study → method name = action

() → no parameters

{ ... } → method body

```
student1.study();    student1.calculateProgress();
```

What Is a Method? — Object Behavior

Method — a named block of code inside a class that performs an action or calculates a result. A method can use object fields, receive input values (parameters),

1. void method
Performs an action but does not return a value.
void displayInfo() { ... }

2. Returning method
Returns a value. Its return type is written instead of void.
double getGpa() { return gpa; }

3. Without parameters
Receives no input values. The parentheses are empty.
void study() { ... }

4. With parameters
Receives input values through parameters declared in ().
void addCredits(int credits) { ... }

```
class Student {
    String name; double gpa; int completedCredits;
    void displayInfo() { System.out.println(name); }
    void addCredits(int credits) { completedCredits += credits; }
    double getGpa() { return gpa; }
    boolean isExcellent(double limit) { return gpa >= limit; }
}
```

Two independent questions:
Does it return a value?
→ void / returning
Does it receive input?
→ no parameters / parameters
Example: isExcellent(double limit)
→ parameter + return value

Classify: boolean isExcellent(double limit)— void or returning? with parameters or without parameters?

Method without Parameters — displayInfo()

Definition. A method without parameters does not receive input values when it is called. Its parentheses are empty: `methodName()`. It can still use the current object's fields and perform useful actions.

```
class Student {  
    String name;  
    int age;  
    double gpa;  
    String major;  
    int completedCredits;  
  
    void displayInfo() {  
        System.out.println("Student: " + name);  
        System.out.println("Age: " + age);  
        System.out.println("Major: " + major);  
        System.out.println("GPA: " + gpa);  
        System.out.println("Credits: " + completedCredits);  
    }  
}
```

Think: Why does `displayInfo()` know which student's data to print?

How does `displayInfo()` work?

1. No arguments

`student1.displayInfo();`

2. Uses object fields

`name`, `age`, `major`, `gpa`

3. Performs an action

prints a student profile

4. Returns no value

return type = `void`

```
student1.displayInfo();  
// method reads student1's own fields
```

Method with Parameters — updateGpa(double newGpa)

Definition. A parameter is a variable declared in a method header. It receives a value when the method is called.
Parameters make a method reusable: the same method can work with different input values.

```
class Student {  
    String name;  
    double gpa;  
    int completedCredits;  
  
    void updateGpa(double newGpa) {  
        gpa = newGpa;  
    }  
  
    void addCredits(int credits) {  
        completedCredits += credits;  
    }  
  
    void printStatus(String message) {  
        System.out.println(name + ": " + message);  
    }  
}
```

Predict: What is student1.gpa after updateGpa(3.9)? What is the argument?

Parameter vs Argument

PARAMETER

double newGpa
Declared in the method definition.

ARGUMENT

3.9
The actual value passed to the method.

student1.updateGpa(3.9);
3.9 → newGpa → gpa
The object's state changes.

```
student1.updateGpa(3.9);  
student1.printStatus("GPA updated");
```

Return Value — double getGpa()

void → method does not return a value.

return value → method must have a return type (int, double, String, etc.).

Definition. A returning method sends a result back to the code that called it. The return type specifies the type of that result. The return statement ends the method and provides the result. The returned value can be stored, printed, compared, or used in another expression.

```
class Student {  
    String name;  
    double gpa;  
    int completedCredits;  
  
    double getGpa() {  
        return gpa;  
    }  
  
    double calculateProgress() {  
        return completedCredits / 240.0 * 100;  
    }  
  
    boolean isExcellent() {  
        return gpa >= 3.5;  
    }  
}
```

Compare: void displayInfo() performs an action; double getGpa() returns a value.

How does return work?

1. Return type → double

The method must return a double value.

2. Method name → getGpa()

No parameters in this example.

3. return statement → return gpa;

Sends the GPA back to the caller.

```
double currentGpa = student1.getGpa();  
System.out.println(currentGpa);  
if (student1.isExcellent()) { ... }
```


Program Demonstration — Objects and Fields

This program demonstrates how one Student class is used to create different objects. Each object stores its own data and uses the same method.

```
class Student {
    String name;
    int age;
    double gpa;
    String major;

    void displayInfo() {
        System.out.println(name + " | " + age);
        System.out.println(major + " | GPA: " + gpa);
    }
}

Student student1 = new Student();
student1.name = "Anna";
student1.age = 20;
student1.gpa = 3.7;
student1.major = "Computer Science";
student1.displayInfo();

Student student2 = new Student();
student2.name = "Alex";
student2.age = 21;
student2.gpa = 3.4;
student2.major = "Software Engineering";
student2.displayInfo();
```

What does the code demonstrate?

1. Class

Student is a template that defines fields and the displayInfo() method.

2. Fields

name, age, gpa, and major describe the state of each Student object.

3. Object creation

new Student() creates a new object. student1 and student2 are separate objects.

4. Dot operator

student1.gpa accesses the gpa field; student1.displayInfo() calls a method.

5. Different state

Anna and Alex have different field values, although they belong to the same class.

6. Same behavior

Both objects can call displayInfo(), but the method prints the data of the object that calls it.

Key idea: ONE CLASS → MANY OBJECTS → SAME STRUCTURE AND METHODS → DIFFERENT STATE

Program Demonstration — Methods, Parameters and Return Values

This program demonstrates how methods receive input values, change an object's state, and return results to the calling code.

```
// Methods inside Student
void updateGpa(double newGpa) {
    gpa = newGpa;
}

double getGpa() {
    return gpa;
}

boolean isExcellent() {
    return gpa >= 3.5;
}

// Using the methods
student1.updateGpa(3.9);
student1.displayInfo();

double currentGpa = student1.getGpa();
System.out.println("Returned GPA = " + currentGpa);

System.out.println(student1.isExcellent());
System.out.println(student2.isExcellent());
```

What does the code demonstrate?

1. Method with a parameter

updateGpa(double newGpa) receives a value that can be used inside the method.

2. Parameter and argument

newGpa is the parameter. In updateGpa(3.9), the value 3.9 is the argument.

3. Changing object state

gpa = newGpa changes the GPA stored in student1 from 3.7 to 3.9.

4. Returning a value

getGpa() has return type double. return gpa sends the value back to the caller.

5. Using the returned value

The returned GPA is stored in the variable currentGpa and can be printed or used later.

6. Boolean method

isExcellent() returns true if GPA $\geq$ 3.5 and false otherwise. Different objects may return dif

Key idea: INPUT → PARAMETER → METHOD → CHANGE / CALCULATION → RETURNED RESULT

Practical Task — Extend the Student Class

Time: 10–15 min

Task:

Extend the existing Student program in the Classes3w project. Do not create a new project. Add new fields and methods, create a third Student object, run the program, and c

Requirements

1. Add two fields

```
String email;  
int completedCredits;
```

2. Add a method with a parameter

```
void addCredits(int credits)  
Increase completedCredits by credits.
```

3. Add a returning method

```
int getCompletedCredits()  
Return the current number of credits.
```

4. Add a boolean method

```
boolean canGraduate()  
Return true if completedCredits >= 240.
```

Use only the Java concepts studied in this lecture.

Create and test student3

```
Student student3 = new Student();  
student3.name = "Maria";  
student3.age = 22;  
student3.gpa = 3.8;  
student3.major = "Computer Science";  
student3.email = "maria@enu.kz";  
student3.completedCredits = 235;
```

```
student3.displayInfo();  
student3.addCredits(10);
```

```
System.out.println(  
    student3.getCompletedCredits());  
System.out.println(  
    student3.canGraduate());
```

Expected check:

235 + 10 = 245 credits → canGraduate() = true

16. What Is a Constructor?

Definition

A constructor is a special member of a class that initializes a new object. It is called automatically when an object is created with the new operator.

Main Rules

1. Same name as the class

For class Student, the constructor is named Student().

2. No return type

A constructor has no return type — not even void.

3. Called automatically

new Student() creates the object and calls Student().

4. Initializes object state

A constructor can assign initial values to fields.

Constructor vs Method

Constructor initializes an object. A regular method performs an action or returns a result after the object exists.

Example — Default Constructor

```
class Student {  
    String name;  
    int age;  
    double gpa;  
  
    Student() {  
        name = "Unknown";  
        age = 18;  
        gpa = 0.0;  
    }  
}  
  
Student student1 = new Student();
```

What happens?

new Student() creates student1 → Student() runs automatically → name = "Unknown", age = 18, gpa = 0.0.

Check: When does Student() execute — when the class is declared or when new Student() is executed?

17. Constructor with Parameters

Definition

A parameterized constructor receives values when an object is created and uses them to initialize the object's fields.

Why Use Parameters?

1. Initialize immediately

The object receives meaningful values at the moment it is created.

2. Reduce separate assignments

We do not need several lines such as `student1.name = ...` after `new`.

3. Create different objects

The same constructor can receive different arguments for Anna, Alex, or Maria.

4. Parameters are local variables

`name`, `age`, and `gpa` exist inside the constructor and receive the arguments.

Important: `this.name = name`

`this.name` is the object's field; `name` is the constructor parameter. The `this` keyword is explained on the next slide.

Example

```
class Student {  
    String name;  
    int age;  
    double gpa;  
  
    Student(String name, int age, double gpa) {  
        this.name = name;  
        this.age = age;  
        this.gpa = gpa;  
    }  
}  
  
Student student1 = new Student("Anna", 20, 3.7);  
Student student2 = new Student("Alex", 21, 3.4);
```

How does it work?

"Anna", 20, 3.7 → constructor parameters → fields of `student1`. The second call creates another object with different parameters.

Check: What values are passed to the constructor in `new Student("Anna", 20, 3.7)`?

Why Do We Use a Constructor?

As we can see from the code, when we create a new object, we have to assign values to its fields separately.

If a class has many fields, this requires a lot of repetitive code and becomes inconvenient. Therefore, we can use a constructor to create and initialize an object. A constructor

Without a Constructor

```
Student student1 = new Student();
student1.name = "Anna";
student1.age = 20;
student1.gpa = 3.7;
```

```
Student student2 = new Student();
student2.name = "Alex";
student2.age = 21;
student2.gpa = 3.4;
```

```
Student student3 = new Student();
student3.name = "Maria";
student3.age = 22;
student3.gpa = 3.8;
```

With a Constructor

```
class Student {
    String name;
    int age;
    double gpa;

    Student(String name, int age, double gpa) {
        this.name = name;
        this.age = age;
        this.gpa = gpa;
    }
}

Student student1 = new Student("Anna", 20, 3.7);
Student student2 = new Student("Alex", 21, 3.4);
Student student3 = new Student("Maria", 22, 3.8);
```

Constructor: less repetitive code → object creation and initialization in one statement → clearer and more convenient code.

20. Method Overloading

Definition

Method overloading means defining several methods with the same name in the same class, but with different parameter lists. Java selects the appropriate method from the available methods.

Rules of Method Overloading

Same method name

All overloaded methods use the same name, for example `displayInfo()`.

Different number of parameters

`displayInfo()` and `displayInfo(boolean showGpa)` are different methods.

Different parameter types

A method can also be overloaded by using different parameter types.

Return type is not enough

Changing only the return type does NOT create a valid overloaded method.

Why is it useful?

We keep one meaningful method name while providing several ways to perform a related operation with different input data.

Example — Overloaded `displayInfo()`

```
class Student {  
    String name;  
    int age;  
    double gpa;  
  
    void displayInfo() {  
        System.out.println(name);  
    }  
  
    void displayInfo(boolean showGpa) {  
        System.out.println(name);  
        if (showGpa) {  
            System.out.println("GPA: " + gpa);  
        }  
    }  
  
    void displayInfo(String message) {  
        System.out.println(message + ": " + name);  
    }  
}
```

Different calls → different method versions

```
student1.displayInfo(); | displayInfo(true); | displayInfo("Student")
```

Key idea: SAME METHOD NAME + DIFFERENT PARAMETERS → JAVA CHOOSES THE MATCHING METHOD

21. Access Modifiers — public, private, protected

Definition

Access modifiers control where a class member can be accessed. They help protect data and define which parts of a program are allowed to use fields, methods, and constructors.

public

Accessible from any class.

```
public void displayInfo()
```

private

Accessible only inside the same class.

```
private double gpa;
```

protected

Accessible in the same package and in subclasses.

```
protected String major;
```

Example in the Student Class

```
public class Student {  
    public String name;  
    private double gpa;  
    protected String major;  
  
    public void displayInfo() {  
        System.out.println(name + " | " + gpa);  
    }  
}
```

Access modifiers define the visibility of class members.

What Can Be Accessed?

public name

Other classes can access student1.name directly.

private gpa

Other classes cannot use student1.gpa directly. Access should be controlled by methods.

protected major

Available in the same package and to subclasses; useful when working with inheritance.

Next: Why should important fields such as gpa be private?

22. Why Do We Use private?

Main idea

We use private to protect an object's internal data from uncontrolled direct access. Other classes cannot change a private field directly; they must use methods provided by the class.

Problem — Direct Access to Data

```
class Student {  
    public double gpa;  
}  
  
Student student1 = new Student();  
student1.gpa = 3.7;    // valid  
student1.gpa = -5.0;   // logically invalid!  
student1.gpa = 12.0;  // logically invalid!
```

Solution — Hide the Field with private

```
class Student {  
    private double gpa;  
  
    public void setGpa(double newGpa) {  
        if (newGpa >= 0.0 && newGpa <= 4.0) {  
            gpa = newGpa;  
        }  
    }  
}
```

1. Data Protection

Prevents direct modification of important fields.

2. Controlled Access

The class decides how its data can be read or changed.

3. Validation

Methods can check values before changing the object's state.

4. Encapsulation

Internal data is hidden behind a clear public interface.

public field → anyone can change it directly | private field → access is controlled by the class
This principle leads us to one of the key ideas of OOP: **ENCAPSULATION**.

23. Encapsulation

Definition

Encapsulation is an OOP principle that combines data and methods inside a class and hides the internal state from uncontrolled direct access. In Java, fields are commonly declared as private.

How Encapsulation Works

1. Hide the data

Declare important fields as private.

2. Provide controlled access

Use public methods to read or change the data.

3. Validate changes

Check new values before updating the object's state.

4. Protect object integrity

The object remains in a valid and predictable state.

Example — Encapsulated Student Class

```
class Student {  
    private String name;  
    private double gpa;  
  
    public Student(String name, double gpa) {  
        this.name = name;  
        setGpa(gpa);  
    }  
  
    public double getGpa() {  
        return gpa;  
    }  
  
    public void setGpa(double gpa) {  
        if (gpa >= 0.0 && gpa <= 4.0) {  
            this.gpa = gpa;  
        }  
    }  
}
```

Outside code cannot directly execute: `student1.gpa = -5.0;`
Instead: `student1.setGpa(3.9);` → the class controls the change. Next: Getter and Setter Methods.

24. Getter Methods

Definition

A getter method is a public method that returns the value of a private field. It provides controlled read access to the object's internal data without allowing direct access to the field.

Structure of a Getter

public

The getter can be called from another class.

double

The return type must match the type of the field.

getGpa()

Common Java naming convention: get + FieldName.

return gpa;

Returns the current value; it does not change the field.

Important

A getter reads data. It does not modify the object's state.

Example — Reading a private Field

```
class Student {  
    private String name;  
    private double gpa;  
  
    public String getName() {  
        return name;  
    }  
  
    public double getGpa() {  
        return gpa;  
    }  
}
```

Using the getter:

```
double currentGpa = student1.getGpa();  
System.out.println("GPA: " + student1.getGpa());
```

private field → getter method → controlled READ access | Next: Setter Methods → controlled WRITE access

25. Setter Methods

Definition

A setter method is a public method that changes the value of a private field. It provides controlled write access to the object's internal data and can check a new value before storing it.

Structure of a Setter

public

The setter can be called from another class.

void

A setter usually changes data and does not return a value.

setGpa(double gpa)

The parameter receives the new value.

this.gpa = gpa;

The field of the current object receives the parameter value.

Important

A setter modifies the object's state. It is also the natural place to add validation.

Example — Changing private Fields

```
class Student {  
    private String name;  
    private double gpa;  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public void setGpa(double gpa) {  
        this.gpa = gpa;  
    }  
}
```

Using setters:

```
student1.setName("Anna");  
student1.setGpa(3.9);
```

26. Validation in a Setter

Definition

Validation is the process of checking a new value before storing it in an object's field. A setter can accept a valid value and reject an invalid one, helping to keep the object in a co

Why Validate Data?

Business rule

For this example, GPA must be between 0.0 and 4.0.

Check before assignment

The if condition tests the parameter before changing the field.

Valid value

The setter stores the new value in this.gpa.

Invalid value

The field remains unchanged and the program reports the problem.

Important

Validation protects the object's state from incorrect values.

Example — Validate GPA before Assignment

```
class Student {  
    private double gpa;  
  
    public void setGpa(double gpa) {  
        if (gpa >= 0.0 && gpa <= 4.0) {  
            this.gpa = gpa;  
        } else {  
            System.out.println("Invalid GPA");  
        }  
    }  
}
```

Test the setter:

```
student1.setGpa(3.9);    // accepted  
student1.setGpa(5.5);    // rejected → "Invalid GPA"
```

INPUT → VALIDATION → if valid: UPDATE FIELD | if invalid: REJECT VALUE Next: Complete Student Class

27. Complete Student Class

Now we combine the main concepts of this lecture in one class: private fields, a constructor, this, methods, getters, setters, and validation.

Student.java

```
public class Student {
    private String name;
    private int age;
    private double gpa;

    public Student(String name, int age, double gpa) {
        this.name = name;
        this.age = age;
        setGpa(gpa);
    }

    public String getName() { return name; }
    public int getAge() { return age; }
    public double getGpa() { return gpa; }

    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }

    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        } else {
            System.out.println("Invalid GPA");
        }
    }

    public void displayInfo() {
        System.out.println(name + " | " + age + " | GPA: " + gpa);
    }
}
```

What Is Used Here?

private fields

name, age, gpa — internal object state

constructor

initializes a new Student object

this

refers to the current object's fields

getters

provide controlled READ access

setters

provide controlled WRITE access

validation

accepts GPA only from 0.0 to 4.0

displayInfo()

defines object behavior

```
Student s1 = new Student("Anna", 20, 3.7);
```

Getter — Reading a private Field from Another Class

A getter allows another class to read the value of a private field.

Class Student — private data

```
public class Student {  
    private String name;  
    private double gpa;  
  
    public String getName() {  
        return name;  
    }  
  
    public double getGpa() {  
        return gpa;  
    }  
}
```

NO — direct access

Teacher cannot use `student.gpa` because `gpa` is private.

Class Teacher — can it read Student's private fields?

```
public class Teacher {  
    public void showStudent(Student student) {  
  
        // Direct access — NOT allowed  
        // System.out.println(student.gpa); // ERROR  
  
        // Access through getter — allowed  
        System.out.println(student.getName());  
        System.out.println(student.getGpa());  
    }  
}
```

YES — through a public getter

Teacher can use `student.getGpa()` if `Student` provides a public getter.

private field → cannot be read directly outside Student → public getter → controlled READ access

Setter — Changing a private Field from Another Class

A setter allows another class to change the value of a private field in a controlled way.

Class Student — private data + setter

```
public class Student {  
    private double gpa;  
  
    public void setGpa(double gpa) {  
        if (gpa >= 0.0 && gpa <= 4.0) {  
            this.gpa = gpa;  
        } else {  
            System.out.println("Invalid GPA");  
        }  
    }  
}
```

NO — direct modification

Teacher cannot use `student.gpa = 3.9` because `gpa` is private.

Class Teacher — can it change Student's private field?

```
public class Teacher {  
    public void updateGpa(Student student) {  
  
        // Direct access — NOT allowed  
        // student.gpa = 3.9; // ERROR  
  
        // Access through setter — allowed  
        student.setGpa(3.9); // OK  
  
        student.setGpa(5.5);  
        // rejected by validation  
    }  
}
```

YES — through a public setter

Teacher can use `student.setGpa(3.9)`. The setter can validate the value first.

private field → cannot be changed directly outside Student → public setter → controlled WRITE access

Practical Task — Complete the Student Class

Task: Enter the final code in IntelliJ IDEA. Create Student.java and Main.java, run the program, and check the output. Time: 10–15 minutes.

1. Student.java

```
public class Student {
    private String name;
    private int age;
    private double gpa;

    public Student(String name, int age, double gpa) {
        this.name = name;
        this.age = age;
        setGpa(gpa);
    }

    public String getName() { return name; }
    public int getAge() { return age; }
    public double getGpa() { return gpa; }

    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }

    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        } else {
            System.out.println("Invalid GPA");
        }
    }

    public void displayInfo() {
        System.out.println(name + " | " + age + " | GPA: " + gpa);
    }
}
```

2. Main.java

```
public class Main {
    public static void main(String[] args) {
        Student student1 = new Student("Anna", 20, 3.7);
        Student student2 = new Student("Alex", 21, 3.4);

        student1.displayInfo();
        student2.displayInfo();

        student1.setGpa(3.9);
        System.out.println("New GPA: " + student1.getGpa());

        student2.setGpa(5.5); // invalid value
    }
}
```

Check after Run:

- Two Student objects are created by the constructor.
- setGpa(3.9) changes Anna's GPA; getGpa() reads it.
- setGpa(5.5) is rejected → "Invalid GPA".

Use: class + private fields + constructor + this + getter + setter + validation + method

Practical Task — Extend the Student Program

TIME: 10–15 min

Continue working with your existing `Student.java` and `Main.java`. Do not create a new project. Add the following elements to your program.

What to Add

1. Make the fields private

Change `name`, `age` and `gpa` to private fields.

2. Add a constructor

Initialize `name`, `age` and `gpa` when a `Student` object is created.

3. Add getters

Create `getName()`, `getAge()` and `getGpa()`.

4. Add setters

Create `setName()`, `setAge()` and `setGpa()`.

5. Add GPA validation

Accept GPA only from 0.0 to 4.0.

6. Test the class

Create two objects, change GPA and read it using `getGpa()`.

Goal: apply encapsulation to the existing `Student` class.

Code Hints

```
private String name;
private int age;
private double gpa;

Student(String name, int age, double gpa) {
    this.name = name;
    this.age = age;
    setGpa(gpa);
}

public double getGpa() {
    return gpa;
}

public void setGpa(double gpa) {
    if (gpa >= 0.0 && gpa <= 4.0)
        this.gpa = gpa;
}
```

Final check:

`setGpa(3.9)` → accepted

|

`setGpa(5.5)` → rejected

Rating 1 – Week 7 — Basic GUI Prototype (50%) without a database

Rating 2 – Week 14 Final Project — Complete Hotel Management GUI (100%) - Admin and user roles; database connectivity implemented

Week 7 — Basic GUI Prototype (50%)

Final Project — Complete Hotel Management GUI (100%)

Week 7 — Rating 1: Hotel Mini-Project GUI

50% COMPLETED

Hotel Project — Required OOP Structure

Parent class: Room

- id
- roomNumber
- capacity
- price

Child classes:

- StandardRoom
- JuniorSuite
- Suite
- + minRentalPeriod
- + maxRentalPeriod

Required methods:

- addRoom()
- displayRooms()

Week 7:

Implement all classes, fields, constructors and main methods

Example GUI — Hotel Room Management

Hotel Room Management System

Room ID

101

Room Number

305

Capacity

2

Price

25000

Room Type

Standard Room

Add Room

Update

Delete

No.	Type	Capacity	Price
305	Standard	2	25,000
410	Junior Suite	3	40,000
501	Suite	4	65,000

Search:

Find

The GUI should allow the user to:

- enter room data
- add / update / delete rooms
- display the room list

Week 7 target: implement the complete OOP class structure + demonstrate a working Hotel GUI = at least 50% of the mini-project.

Rating 2 – admin and user roles, plus database connectivity implementation

Week 7 — Basic GUI Prototype (50%)

Final Project — Complete Hotel Management GUI (100%)



City Hotel

Hotel Management System

Tuesday, September 30, 2026

11:24

Administrator

Dashboard

Rooms

Reservations

Guests

Check-in / Check-out

Services

Reports

Settings

Room Management

Room Types

Reservations

Guests

Check-in / Check-out

Reports

Room Information

Room ID: R001

Room Number: 101

Room Type: Standard Room

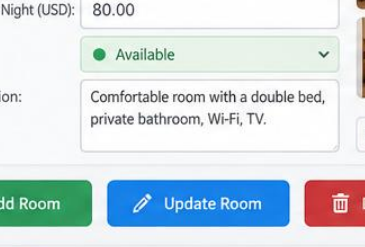
Floor: 1

Capacity: 2

Price per Night (USD): 80.00

Status: Available

Description: Comfortable room with a double bed, private bathroom, Wi-Fi, TV.



Add Photo

Remove Photo

View

Search and Filter

Room Type: All

Status: All

Floor: All

Min. Price:

Max. Price:

Search

Clear

Check Availability

Check-in Date: 30.09.2026

Check-out Date: 02.10.2026

Guests: 2

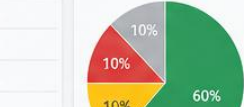
Check Availability

Room Type Statistics



Room Type	Percentage
Standard Room	50%
Junior Suite	30%
Suite	20%

Room Status Statistics



Status	Percentage
Available	60%
Occupied	10%
Under Cleaning	10%
Maintenance	10%

List of Rooms

ID	Room Number	Type	Floor	Capacity	Price (USD)	Status	Description
R001	101	Standard Room	1	2	80.00	Available	Comfortable room with a double bed ...
R002	102	Standard Room	1	2	80.00	Available	City view, Wi-Fi, TV
R003	201	Junior Suite	2	3	120.00	Occupied	Spacious room with a sofa ...
R004	202	Standard Room	2	2	80.00	Available	Quiet room, garden view
R005	301	Suite	3	4	200.00	Under Cleaning	Large room with living area ...
R006	302	Junior Suite	3	3	120.00	Available	Balcony, city view
R007	401	Suite	4	4	200.00	Available	Panoramic view, living room ...
R008	402	Standard Room	4	2	80.00	Maintenance	Room under maintenance
R009	501	Junior Suite	5	3	120.00	Available	Modern interior
R010	502	Standard Room	5	2	80.00	Available	Business room, workspace

Week 7 — Rating 1

Week 7 — Rating 1

Complete at least 50% of the mini-project.

Implement all required **classes, fields, constructors, and methods.**

Implement **inheritance and interaction between classes.**

Create a working **GUI (window application).**

Implement the main forms, buttons, input fields, tables, and basic operations.

Demonstrate that the application starts and the main functions work.

Database connection is NOT required for Rating 1.

Rating 1 → Working OOP model + GUI, without a database.

Rating 2 → Admin and User roles + Database connectivity and data operations.

Peter Salzmann | Wingsuit & Innovation Showreel 2026



Peter Salzmann

Austrian athlete and wingsuit innovator.

Flight Duration

Peter Salzmann spent about 160 seconds in the air during the flight.

The Flight

Peter Salzmann completed an unusual flight in a special wingsuit. He controlled the flight by moving his body. The aerodynamic wing allowed him to glide and use air currents. The flight demonstrated new possibilities for individual flight technology.

Video: Peter Salzmann — Wingsuit & Innovation Showreel 2026

Key idea:

A wingsuit increases the aerodynamic surface of the body and makes controlled gliding possible.

Wingsuit + a special rigid aerodynamic wing (foil)

This combination makes the flight look as if a person is flying like a bird.

How Is This Kind of Flight Filmed?

Several methods can be used: cameras on the ground or mountains, cameras mounted on the athlete, and fast FPV drones flying nearby. In some wingsuit projects, professional camera operators also fly beside the athlete with helmet-mounted cameras.

Technology: wingsuit • aerodynamic foil • body control • FPV filming

Peter Salzmann | Wingsuit & Innovation Showreel 2026



Петер Зальцманн (Peter Salzmann)

Австрийский спортсмен Петер Зальцманн провел в воздухе сто шестьдесят секунд на парплане

Австрийский спортсмен **Петер Зальцманн (Peter Salzmann)** совершил необычный полёт в специальном **костюме-крыле (wingsuit)**. Он провёл в воздухе около **160 секунд**, управляя полётом движениями своего тела. Благодаря специальной конструкции крыла спортсмен мог планировать и использовать воздушные потоки, подобно птице. Полёт продемонстрировал новые возможности современных технологий индивидуального полёта.

<https://www.youtube.com/watch?v=s1LPCWoQbpM&t=4s>

https://yandex.kz/video/preview/6443857797623167843?q_source=title

wingsuit + специальное жёсткое аэродинамическое крыло (foil). Именно поэтому этот полёт выглядит так, будто человек действительно летит как птица.

А как снимают такой полёт?

Очень интересно: обычно применяют **несколько способов съёмки**. Это камеры с земли/с горы, камеры на самих спортсменах или операторах и быстрые **FPV-дроны**, которые летят рядом. В одном из недавних рекордных полётов Зальцманна его сопровождал специально подготовленный высокоскоростной дрон, который снимал полёт. В других wingsuit-проектах Red Bull используются даже профессиональные операторы в wingsuit: они летят рядом со спортсменами **с камерой на шлеме**.

Why Is the Wingsuit Foil Project Relevant to Modern IT?

A real example of digital engineering: from a virtual model and computer simulation to a tested physical flying system

RELEVANCE

The project shows how IT can solve a complex engineering problem: increase lift and glide performance, optimize a new aerodynamic foil, reduce risky trial-and-error, and verify the design through prototypes, wind-tunnel tests and real flights.

IT / Engineering Technology	How It Was Used	What It Helped Optimize	Result in the Project
CAD Computer-Aided Design	Digital drawings and precise geometry of the foil were created by Red Bull Racing / Red Bull Technology.	Foil shape, dimensions, mounting geometry and component design.	A manufacturable digital design before building the physical wing.
CFD Computational Fluid Dynamics	Computer simulations modeled airflow around the foil and supported aerodynamic development.	Foil profile, airflow, lift, drag, angle and efficient wing configuration.	More informed design decisions before dangerous real-flight testing.
Digital Prototyping	Six prototypes were developed; Salzmann reports about 10 configurations for each prototype.	Geometry, glide ratio, chest mount, mast, connectors and wing position.	Fast iterative improvement: design → test → modify → test again.
3D Printing	Selected 3D components of the final foil system were produced additively.	Complex small parts and rapid component iteration.	Faster fabrication of prototype/final components.
IDEA → CAD MODEL → CFD SIMULATION → PROTOTYPE → 3D PRINTING → WIND TUNNEL → REAL FLIGHT			

OVERALL CONCLUSION: Modern IT connected virtual engineering with the physical world — computer models guided the design, prototypes validated it, and real-flight tests demonstrated the final technology.

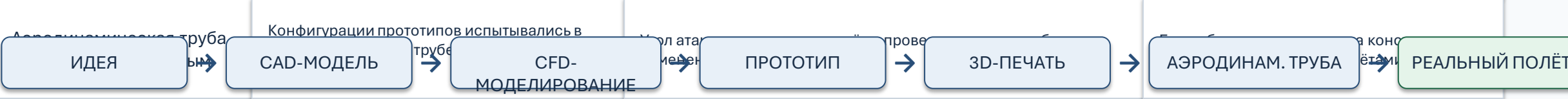
Почему проект Wingsuit Foil актуален для современных ИТ?

Реальный пример цифровой инженерии: от виртуальной модели и компьютерного моделирования к испытанной физической системе полёта

АКТУАЛЬНОСТЬ

Проект показывает, как ИТ помогают решать сложную инженерную задачу: повышать подъёмную силу и эффективность планирования, оптимизировать новое аэродинамическое крыло, сокращать рискованные испытания методом проб и ошибок и проверять конструкцию с помощью прототипов, аэродинамической трубы и реальных полётов.

ИТ / инженерная технология	Как использовалась	Что помогла оптимизировать	Результат в проекте
CAD Computer-Aided Design	Создавались цифровые чертежи и точная геометрия аэродинамического крыла.	Форма и размеры крыла, геометрия крепления и конструкция компонентов.	Готовая цифровая конструкция до изготовления физического крыла.
CFD Computational Fluid Dynamics	Компьютерное моделирование воспроизводило воздушные потоки вокруг крыла и поддерживало аэродинамическую разработку.	Профиль крыла, воздушный поток, подъёмная сила, сопротивление, угол и эффективная конфигурация.	Более обоснованные инженерные решения до рискованных реальных полётов.
Цифровое прототипирование	Было разработано шесть прототипов; для каждого испытывались различные конфигурации.	Геометрия, качество планирования, крепление, стойка, соединения и положение крыла.	Быстрое итерационное улучшение: проектирование → тест → изменение → повторный тест.
3D-печать	Отдельные компоненты финальной системы изготавливались с помощью аддитивных технологий.	Сложные небольшие детали и быстрое изменение компонентов.	Более быстрое изготовление прототипов и финальных компонентов.



ОБЩИЙ ИТОГ: современные ИТ связали виртуальное проектирование с физическим миром — компьютерные модели направляли разработку, прототипы позволяли проверять решения, а реальные полёты демонстрировали итоговую технологию.